

SAÉ CyberDiag

Veille technologique



Table des matières :

I.	Introduction.....	2
II.	Critères de comparaison	2
III.	Solutions répondant au besoin	3
1.	Application (back-end)	3
2.	Interface graphique (front-end)	4
3.	Sauvegarde des données.....	5
4.	Plus de détail sur notre format propriétaire (.cbdg)	7
5.	Documentations :	9
IV.	Conclusion sur les solutions.....	10

Un projet par :

- TOUZEAU Célian
- HUBERT Mathis
- LÉTANG Augustin
- MONNOT Yohan
- BLANCHETIÈRE--DRÔLON Nolhan

I. Introduction

Dans un contexte où les petites structures disposent rarement de moyens techniques ou humains suffisants pour assurer une évaluation régulière de leur niveau de cybersécurité. Il est essentiel de s'appuyer sur des solutions accessibles, automatisées et adaptées à leurs besoins. Les technologies que nous allons utiliser doivent ainsi répondre à certains critères. Nous les avons listés ci-dessous, afin de garantir la pertinence et la faisabilité des solutions retenues dans le cadre du projet.

Le but de ce document est de fournir et de pousser à une réflexion approfondie quant aux différentes solutions techniques dans la mise en place de notre logiciel d'audit système et réseau.

II. Critères de comparaison

- **Facilité de mise en œuvre** : Simplicité et rapidité à déployer ou intégrer la technologie dans un environnement existant.
- **Efficacité** : Capacité à atteindre les résultats souhaités avec un minimum de ressources (temps, coût, énergie).
- **Correspondant au besoin** : Adéquation précise entre les fonctionnalités de la technologie et les exigences du projet.
- **Portabilité** : Aptitude à être transférée ou réutilisée dans un autre environnement sans modification majeure.
- **Facilité de maintenance** : Simplicité à corriger, mettre à jour ou faire évoluer la technologie après son déploiement.
- **Automatisation des tâches** : Capacité à réduire l'intervention manuelle en automatisant des processus répétitifs.
- **Extensibilité** : Peut-elle évoluer avec vos besoins futurs ?
- **Intégration** : Compatibilité avec les outils et systèmes existants (API, protocoles, formats de données).
- **Performance** : Temps de réponse, capacité à gérer la charge, latence, etc.
- **Sécurité** : Historique des vulnérabilités, politiques de sécurité, chiffrement, gestion des accès.
- **Benchmarking** : Comparaison avec des technologies concurrentes (performances, coûts, etc.).

Des critères de comparaison spécifiques pour la sauvegarde des données et la documentation ont été définis dans leur sous-partie respectives.

III. Solutions répondant au besoin

1. Application (back-end)

Nous cherchons une technologie pour la réalisation des tâches en arrière-plan (diagnostics, traitement des données, etc), nous appelons cela le back-end d'une application, pour cela, nous comparons différentes technologies.

Technologies comparées : Bash, C, Java, Python, Node.js

Technologie	Bash	C	Java	Python	Node.js
Langage	Script Shell	C	Java	Python	JavaScript (Express, NextJS)
Facilité de mise en œuvre	Prototypage rapide, tâches simples système	Nécessite compilation et gestion mémoire	Moyennement facile, nécessite configuration	Très facile, syntaxe simple	Facile asynchrone, rapide à développer
Efficacité	Faible : interprété, peu optimisé	Très élevée : rapide proche du matériel	Bonne : performante après warm-up	Moyenne : interprété, bon pour scripts	Bonne : rapide, mais moins bon pour calculs lourds
Correspondant au besoin	Idéal pour scripts simples	Idéal pour logiciels performants ou bas niveau	Application entreprise, mobiles, backends	Scripts, data science, IA, web API	Application web temps réel, API, microservices
Portabilité	Bonne sur Unix/Linux moins sur Windows	Fonction quasiment partout	Très portable (via JVM) mais Java bloque certaines opérations	Très portable	Très portable
Facilité de Maintenance	Scripts courts faciles à modifier	Nécessite recompilation, plus de rigueur	Code structuré, facile à maintenir	Très lisible, bien structuré	Modulaire, bonne lisibilité
Automatisation des tâches	Excellent pour tâches système et DevOps	Trop bas niveau pour ce domaine.	Possible avec framework spécifique	Excellent : scripting, outils, data pipelines	Bonne : avec scripts Node ou outils npm
Extensibilité	Difficile à faire évoluer	Très bonne si structuré dès le départ	Excellente : OOP, modulaire	Très bonne : code modulaire, extensible	Très bonne : avec npm et modules
Intégration	Excellent avec l'environnement système	Bonne dans des systèmes logiciels	Bonne : avec écosystème Java	Très bonne : intégration	Excellente : API REST,

				facile via package	microservice, JS fullstack
Performance	Lent car interprété	Excellente : bas niveau, très rapide	Bonne : JIT améliore les perfs	Moyenne : dépend de l'usage	Bonne : asynchrone, non-bloquant
Sécurité	Vulnérable aux injections si mal écrites	Faibles mémoire fréquentes si mal gérées	Bonne sécurité si bien configurée	Moyenne : dépend du développeur	Sensible aux injections si mauvaises pratiques
Benchmarking	Pratique pour orchestrer des tests externes	Très précis pour mesurer les performances internes	Outils de profiling disponibles	Facile à mettre en place avec modules	Possible via modules

Nous avons choisi que l'application soit développée en Bash pour pouvoir être utilisée dans un terminal dans un premier temps.

2. Interface graphique (front-end)

Nous cherchons une technologie pour l'affichage de l'interface utilisateur, nous appelons cela le front-end d'une application, pour cela, nous comparons différentes technologies.

Technologies comparées : Java, Python, Web

Technologie	Java	Python	Web
Langage	Java	Python	JavaScript / TypeScript / PWA / Flutter / React / Vue.js / Angular
Facilité de mise en œuvre	Configuration initiale nécessaire, code structuré et typé.	Prototypage rapide, syntaxe simple et claire.	Mise en œuvre rapide, outils et frameworks accessibles.
Efficacité	Très élevée : compilé en bytecode, rapide et stable.	Correcte : interprété, mais optimisable via bibliothèques.	Moyenne : dépend du navigateur et du moteur JavaScript.
Correspondant au besoin	Idéal pour les applications complexes et backend robustes.	Polyvalent : data, IA, web, automatisation.	Parfait pour l'interface utilisateur et les services web.
Portabilité	Fonctionne sur toute JVM, multiplateforme.	Exécution sur la plupart des systèmes.	Compatible avec tous les navigateurs.

Facilité de Maintenance	Code long mais structuré, maintenance rigoureuse.	Code court, lisible et facilement modifiable.	Dépend du framework, séparation front/back parfois complexe.
Automatisation des tâches	Possible mais peu pratique, plus orienté serveur.	Excellent pour scripts et tâches système.	Limité à l'automatisation côté client (navigateur).
Extensibilité	Très extensible via bibliothèques et frameworks (Spring).	Extensible grâce à de nombreux modules Python.	Extensible par APIs, frameworks JS et WebAssembly.
Intégration	Bonne intégration avec bases de données et services.	Intégration facile avec d'autres langages et outils.	Intégration native via HTTP, REST et APIs.
Performance	Performances stables, proche du matériel.	Plus lent, mais suffisant pour la plupart des usages.	Variable selon le navigateur et la complexité du code.
Sécurité	Sécurisé via JVM et gestion stricte des types.	Dépend des bibliothèques utilisées et du développeur.	Peut être vulnérable sans bonnes pratiques (XSS, CSRF).
Benchmarking	Outils puissants : JMH, VisualVM.	Tests et profils simples : pytest-benchmark, cProfile.	Outils intégrés : Chrome DevTools, Lighthouse.

Nous avons choisi que l'interface graphique soit développée en Java pour pouvoir en faire une application.

3. Sauvegarde des données

Critères de choix supplémentaires :

- **Structure des données** : Structurées (tables, relations) ou non structurées (documents, clés/valeurs).
- **Volume et performance** : Taille des données, fréquence d'accès, besoin de requêtes complexes.
- **Intégration avec le back-end** : API, librairies, ou protocoles de communication.

Nous cherchons une technologie pour la sauvegarde des données de l'application, pour cela, nous comparons différentes technologies.

Technologies comparées : JSON, SQL, NoSQL, Fichiers plats

Technologie	Format propriétaire	JSON	SQL	NoSQL	Fichiers plats
Langage	.cbdg (voir section « Notre format »)	JavaScript Object Notation	Structured Query Language	MongoDB, Redis	CSV, XML, YAML
Structure des données	Personnalisable (ex: binaire ou texte structuré)	Non structuré / Semi-structuré	Structuré (tables, relations)	Non structuré (documents, clés/valeurs)	Structuré / Semi-structuré
Facilité de mise en œuvre	Modérée (développement spécifique nécessaire)	Très facile	Modérée (nécessite schéma)	Facile	Très facile
Efficacité	Excellente (format optimisé pour la performance)	Bonne pour petits volumes	Excellente pour requêtes complexes	Bonne pour gros volumes	Limitée (pas de requêtes)
Correspondant au besoin	Idéal pour données sensibles et performances	Idéal pour échanges légers	Parfait pour données relationnelles	Adapté aux données flexibles	Adapté aux données statiques
Portabilité	Limitée (nécessite un lecteur dédié)	Très haute (universel)	Haute (avec SGBD standard)	Moyenne (dépend du système)	Très haute
Facilité de Maintenance	Bonne (si bien documenté et outils fournis)	Haute (pas de schéma)	Modérée (schéma rigide)	Haute (schéma flexible)	Haute
Automatisation des tâches	Bonne (scripts/outils dédiés nécessaires)	Facile (parsing)	Excellente (requêtes SQL)	Bonne (API/drivers)	Facile (scripts)
Extensibilité	Contrôlée (nécessite mise à jour du format)	Moyenne (ajout de champs)	Modérée (modification de schéma)	Excellente (schéma dynamique)	Faible (fichiers)
Intégration	Moyenne (nécessite API ou SDK dédié)	Très facile (bibliothèques JSON)	Facile (ORM/drivers SQL)	Facile (API spécifiques)	Très facile (lecture/écriture)

Performance	Excellente (binaire compact et optimisé)	Bonne pour petits jeux de données	Excellente (optimisée)	Excellente (scalabilité)	Faible (lecture complète nécessaire)
Sécurité	Excellente (chiffrement natif, signature)	Faible (texte brut)	Excellente (SGBD sécurisés)	Variable (dépend du système)	Faible
Benchmarking	Excellente (lecture/écriture rapide)	Moins pertinent (pas de requêtes)	Excellente (outils de benchmark)	Bonne (tests de scalabilité)	Non applicable
Volume et performance	Excellente (binaire compact, efficace)	Limité aux petits/moyens volumes	Excellente pour gros volumes	Excellente pour très gros volumes	Limité aux petits volumes
Intégration avec le back-end	Moyenne (nécessite outils spécifiques)	Très facile (API, librairies)	Facile (connecteurs SQL)	Facile (API/drivers)	Très facile (fichiers)

Nous avons choisi que les données seront sauvegardées avec notre propre format (.cbdg).

4. Plus de détails sur notre format propriétaire (.cbdg)

1. Présentation

Le format **.cbdg** est un format de fichier propriétaire conçu pour stocker les résultats de diagnostics de sécurité système et réseau. Il combine **chiffrement, intégrité et structure normalisée** pour répondre aux exigences de sécurité des collectivités locales et entreprises.

2. Structure du fichier

Le fichier est divisé en 3 sections :

A. En-tête (HEADER)

Contient les métadonnées du diagnostic (Version du format, horodatage, signature numérique (SHA-256), niveau de diagnostic, type de diagnostic, statut).

B. Données chiffrées (ENCRYPTED_DATA)

Contient les résultats du diagnostic au format JSON chiffré (AES-256-GCM)

C. Pied de page (FOOTER)

Contient le checksum (CRC32) pour vérifier l'intégrité du fichier

3. Fonctionnement

A. Chiffrement & Sécurité

- **Chiffrement AES-256-GCM** pour la confidentialité des données.
- **Signature SHA-256** pour garantir l'intégrité.
- **Format binaire compact** (basé sur Protobuf) pour une lecture/écriture rapide.

B. Intégration des recommandations ANSSI

- Chaque vulnérabilité est associée à une **référence ANSSI** (ex: ANSSI-2023-01).
- Priorisation des actions selon la **criticité** (ex: CVE-2023-24932 - Critique).

4. Exemple d'implémentation

```
[HEADER]
Version: 1.0
Timestamp: 2025-10-21T12:05:00Z
Signature: xyz789uvw012...
Diagnostic_Level: advanced
Diagnostic_Type: system
Status: success

[ENCRYPTED_DATA]
{
  "system_info": {
    "os": "Debian 11.7",
    "kernel": "5.10.0-26-amd64",
    "cpu": "Intel Xeon E5-2630 v4",
    "memory": {
      "total": "32GB",
      "used": "18GB"
    }
  },
  "security_analysis": {
    "module": "system_info": {
      "vulnerable_packages": [
        {
          "name": "openssl",
          "installed_version": "1.1.1n-0+deb11u4",
          "fixed_version": "1.1.1n-0+deb11u5",
          "cve": "CVE-2023-0286",
          "severity": "high",
          "recommendation": "ANSSI-2023-06: Update OpenSSL immediately"
        }
      ]
    },
    "module": "basic_security": {
      "insecure_configurations": [
        {
          "file": "/etc/ssh/sshd_config",
          "issue": "PasswordAuthentication enabled",
          "recommendation": "ANSSI-2023-07: Disable password authentication for SSH"
        }
      ]
    },
    "module": "update_check": {
      "network_services": [
        {
          "service": "ftp",
          "status": "running",
          "port": 21,
          "recommendation": "ANSSI-2023-08: Replace with SFTP/SCP"
        }
      ]
    }
  },
  "module_list": ["system_info", "basic_security", "update_check"]
}

[FOOTER]
Checksum: 5e6f7g8h
```

5. Documentations :

Critères de choix :

- **Intégration dans l'application** : Facilité à embarquer ou afficher la documentation.
- **Interactivité** : Possibilité d'ajouter des liens, des images, des vidéos, ou des éléments dynamiques.
- **Accessibilité** : Lisibilité et adaptabilité (ex : mode sombre, taille de police).

Nous cherchons une technologie pour la documentation, pour cela, nous comparons différentes technologies. Cette documentation n'est pas générée par l'application, mais par les développeurs, elle est donc statique.

Technologies comparées : PDF, HTML, Markdown, RST, Wiki

Technologie	PDF	HTML	MarkDown	ReStructuredText (RST)	Wiki (DokuWiki, MediaWiki)
Facilité de mise en œuvre	Modérée (outils de génération), un fichier par documentation	Modéré (avec éditeurs WYSIWYG), une page par documentation	Facile (syntaxe simple), un fichier par documentation	Modérée (syntaxe complexe)	Facile (interface web)
Efficacité	Bonne pour l'impression	Excellente (interactive)	Bonne (conversion facile)	Bonne (documentation technique)	Excellente (collaborative)
Correspondant au besoin	Idéal pour documents officiels	Parfait pour tutoriels interactifs	Adapté aux docs techniques	Idéal pour docs logicielles	Adapté aux docs collaboratives
Portabilité	Très haute	Très haute	Très haute	Haute	Moyenne (nécessite serveur)
Facilité de Maintenance	Faible (régénération nécessaire)	Excellente (fichiers modifiables)	Excellente (fichiers texte)	Bonne (fichiers texte)	Bonne (interface web)
Automatisation des tâches	Moyenne (génération via outils)	Excellente (scripts, CMS)	Excellente (outils comme MkDocs)	Bonne (Sphinx)	Moyenne (extensions)
Extensibilité	Faible (format fixe)	Excellente (CSS, JS)	Bonne (extensions Markdown)	Bonne (directives RST)	Excellente (plugins)

Intégration	Moyenne (visualiseurs PDF)	Très facile (WebView)	Facile (conversion HTML)	Facile (Sphinx)	Difficile (intégration serveur)
Performance	Bonne (lecture rapide)	Excellente (optimisation web)	Excellente (léger)	Bonne	Variable (dépend du serveur)
Intégration dans l'application	Difficile (nécessite visualiseur)	Très facile (WebView)	Facile (conversion HTML)	Facile (HTML via Sphinx)	Difficile (nécessite serveur)
Accessibilité	Haute (standards PDF)	Excellente (CSS, ARIA)	Moyenne (dépend du rendu)	Moyenne	Excellente (thèmes adaptables)

Nous avons choisi que la documentation sera écrite sous le format PDF afin de faciliter sa lecture et son ergonomie auprès des utilisateurs.

IV. Conclusion sur les solutions

À l'issue de notre étude comparative, plusieurs technologies ont été analysées selon des critères objectifs tels que la facilité de mise en œuvre, la performance, la sécurité, l'intégration, ou encore l'adéquation au besoin.

Les solutions retenues sont donc les suivantes :

- **Bash** pour l'application (backend), cette technologie a été retenue pour sa simplicité, sa compatibilité avec l'environnement Linux et sa capacité à automatiser efficacement les tâches système.
- **Java** pour l'interface graphique, cette technologie a été retenue pour son efficacité, son intégration avec base de données et avec les services.
- **Format propriétaire (.cbdg -> pour CyberDiag)** pour la sauvegarde des données, cette technologie a été retenue pour sa structure de données personnalisable, son excellente efficacité et son excellente sécurité.
- **PDF** pour la documentation, cette technologie a été retenue pour sa stabilité et sa portabilité universelle.